

University of Dhaka

Department of Computer Science and Engineering

CSE 2206 – Microcontroller and Embedded System

Lab Assignment 4

ADC Multi-Resolution Testing with Flash-Based Data Logging

Flash Implementation Report

Submitted to

Dr. Mosaddek Hossain Kamal (Tushar)

Professor
Dept. of CSE, DU

Submitted by

MD.Jahidul Islam Sarker

Roll: 01
Registration No.: 2023015944
Lab Group : B11

01-09-2026

1 Objective & Summary

The objective of this work is to implement and verify the **Flash memory portion** on the STM32F446RE using both register-level (bare-metal) code and the STM32 HAL. The implementation provides Flash unlock/lock, sector erase, 32-bit word programming, memory-mapped reading, read-back verification, one-time student identity provisioning, persistent result storage, boot-time display, and an any-byte UART trigger.

Scope : The implemented scope is limited to Flash-memory functionality. The four displayed voltages are deterministic simulated values used to verify Sector 7 storage and retrieval. ADC1, PA0, multi-resolution conversion, raw-sample logging, and real ADC averaging are outside the scope of this implementation.

Both bareMetal and hal reserve Sectors 6 and 7 from application code by limiting the linker **FLASH** region to 256 KB. The terminal evidence records successful Flash self-test, identity provisioning, rejection of repeated provisioning, blank-data handling, result replacement, reset persistence, and UART burst debouncing.

2 System Design Overview

Flash erase is **sector-granular**: erasing a sector restores all bits in that sector and removes every stored byte in it. the implementation therefore separates the write-once student identity in Sector 6 from the replaceable Flash-only result record in Sector 7. Both sectors are non-volatile and are excluded from application placement by setting the linker **FLASH** length to 256 KB.

2.1 Flash Sector Allocation

Sector	Base address	Size	Purpose	Validity marker
6	0x08040000	128 KB	Student identity: written once by the dedicated provisioning build and not erased during normal tests.	0xB1010001
7	0x08060000	128 KB	Flash-only result record: erased and replaced after each accepted UART trigger.	0xCAFEBAFE

2.2 On-Flash Data Records

The baremetal and HAL implementations use the following data definitions. The identity occupies 64 bytes (16 words), while the result record occupies 20 bytes (5 words). In this Flash implementation, simulated voltages are stored as unsigned integer millivolts rather than `float` values.

```

typedef struct {
    uint32_t marker;
    char registration[16];
    char roll[12];
    char name[32];
} StudentInfo_t;           /* 64 bytes */

typedef struct {
    uint32_t marker;
    uint32_t mv12;
    uint32_t mv10;
    uint32_t mv8;
    uint32_t mv6;
} ResultsBlock_t;         /* 20 bytes */

```

The program converts each structure to a word array through two union types:

`StudentInfoImage_t` and `ResultsImage_t`.

This allows aligned 32-bit Flash programming. For both records, payload words are written and verified first; the validity marker is written last as the commit word. An erased marker is `0xFFFFFFFF`, while the TC2/TC3 self-test pattern is `0xDEADBEEF`.

2.3 Stored Result-Field Interpretation

The four result fields preserve the 12, 10, 8, and 6 bit labels, but the Flash-only program does not configure or sample ADC1. The `RunFlashOnlyResultsDemo()` routine generates the values solely to verify Sector 7 erase, write, replacement, read-back, and persistence.

Field	C type	Unit	Generation rule
mv12	uint32_t	millivolts	<code>base_mv</code>
mv10	uint32_t	millivolts	<code>base_mv + 1</code>
mv8	uint32_t	millivolts	<code>base_mv + 6</code>
mv6	uint32_t	millivolts	<code>base_mv + 25</code>

Scope: These four labels and stored millivolt fields exercise the Flash data layout only. They are not measured ADC samples and do not demonstrate ADC resolution, averaging, or quantisation accuracy.

2.4 Boot Sequence and Result Update

At normal boot, firmware performs the following sequence:

1. Read and display the Sector 6 identity block first.
2. Read the Sector 7 marker and display stored results, or print “No previous test data” when the marker word is erased.
3. Wait for any byte on USART2 at 115200 baud, then consume the remaining bytes in the 50 ms quiet window.
4. Generate the four deterministic Flash-test millivolt values, erase Sector 7, verify the erased marker, write the payload and marker, verify the full block, and display the newly stored values.

3 Flash Utility Implementation and Verification

The bare-metal project directly uses `FLASH->KEYR`, `FLASH->CR`, and `FLASH->SR`. It waits for `BSY` to clear, configures sector erase or 32-bit programming, checks controller error flags, disables the active Flash operation, re-locks the controller, and verifies programmed data by direct read-back. The HAL project implements the same operations with `HAL_FLASH_Unlock()`, `HAL_FLASHEx_Erase()`, `HAL_FLASH_Program()`, and `HAL_FLASH_Lock()`. Both versions restrict writes to aligned addresses within Sectors 6 and 7.

The dedicated self-test build is selected by setting `FLASH_SELF_TEST_BUILD` to 1 while keeping the provisioning build disabled. It erases Sector 7 and confirms `0xFFFFFFFF` at its base (TC2), programs `0xDEADBEEF` and reads it back (TC3), and finally erases Sector 7 again so the normal blank-results condition can be tested. The terminal output reports:

```
TC2/TC3 Flash self-test:  OK
```

The self-test intentionally erases Sector 7. It never erases Sector 6. Provisioning, self-test, and normal operation are separate builds; the two special build flags must not be enabled simultaneously.

4 Identity-Provisioning Process

Registration number, roll number, and name are stored once in Sector 6 through the dedicated build flag `FLASH_PROVISION_IDENTITY_BUILD`. The identity data is defined in `student_image`. The provisioning procedure is:

1. Set `FLASH_PROVISION_IDENTITY_BUILD` to 1 and keep `FLASH_SELF_TEST_BUILD` at 0.
2. Build and flash the provisioning firmware.
3. If Sector 6 doesn't already contain the valid identity marker, erase Sector 6, verify the erased marker, write the identity payload, write the marker last, and verify the complete 64-byte block.
4. Observe **Provisioning: OK** and confirm the displayed identity.
5. Set `FLASH_PROVISION_IDENTITY_BUILD` back to 0, rebuild, and flash the normal application using sector erase rather than mass erase.

If the routine is run again after successful provisioning, it detects `0xB1010001` and returns identity already provisioned without erasing or rewriting Sector 6. Terminal Cases 2 and 3 both behaviours.

5 Test-Trigger UX and Results Logging

USART2 uses PA2 (TX), PA3 (RX), **115200 baud**, 8 data bits, no parity, and one stop bit. No specific command character is required. In the bare-metal version, the main loop polls `USART2->SR` for `RXNE`; in the HAL version it calls `HAL_UART_Receive()`.

After the first received byte, a 50 ms quiet-window flush consumes additional bytes from the same burst. The update begins only after the input has remained quiet for 50 ms, preventing duplicate or overlapping Sector 7 updates. In this Flash-only build, each accepted trigger generates four deterministic simulated millivolt values, erases Sector 7, writes the result payload, commits the marker last, verifies the complete block, and prints **Sector 7 update: OK**.

6 Persistence and Robustness

Both records are read directly from their fixed Flash addresses at every boot, so valid identity and result data survive a reset without being copied from RAM. Result replacement erases Sector 7 only; Sector 6 is never touched by the normal logging path. programming the marker last prevents an incomplete payload from being accepted as a committed record after an interrupted write. for stronger power-failure tolerance, two alternating result slots with sequence numbers and checksums could preserve the newest complete copy while the other copy is updated.

The linker reservation is also part of this protection.both projects limit application Flash to 0x08000000--0x0803FFFF. Consequently, the linker cannot place program code or constants in Sectors 6 or 7. When reflashing after provisioning, sector erase must be used; a full chip mass erase would intentionally remove both records.

7 Test Cases (TC1–TC12)

ID	Test case	Flash-only action	Actual/observed result	Verdict
TC1	Raw ADC responsiveness	Outside the Flash.	ADC1 and PA0 are not configured; therefore, raw ADC response was not measured.	Not tested
TC2	Flash erase verification	Dedicated self-test erases Sector 7 and reads 0x08060000.	The erased word matched 0xFFFFFFFF; terminal reported TC2/TC3 Flash self-test: OK.	Pass
TC3	Flash write/read-back	Program 0xDEADBEEF at the Sector 7 base and read it back.	Direct read-back matched 0xDEADBEEF; self-test reported OK.	Pass
TC4	Identity provisioning	Run the dedicated provisioning build and verify the 64-byte identity block.	Marker, registration 2023015944, roll 01, and name MD.Jahidul Islam Sarker were displayed; first provisioning reported OK. A repeated attempt was rejected.	Pass
TC5	Identity display at boot	Reset after provisioning and observe boot order.	Identity was displayed before the Sector 7 results and application prompt.	Pass
TC6	Identity survives testing	Trigger multiple Sector 7 updates and reset.	Sector 6 identity remained unchanged while only Sector 7 results changed.	Pass
TC7	Blank results fallback	Leave Sector 7 erased and start normal mode.	Firmware displayed “No previous test data” and continued waiting for UART input.	Pass
TC8	Per-resolution averaging	Outside the Flash-only implementation.	No ADC samples or independently calculated ADC averages were produced; simulated values test Flash storage only.	Not tested
TC9	Result overwrite on re-test	Send two separate UART triggers and reset after the second update.	First stored set was 1.047/1.048/1.053/1.072 V; second set was 1.394/1.395/1.400/1.419 V. After reset, only the second set was displayed.	Pass
TC10	Resolution boundary sanity	Outside the Flash-only implementation.	The displayed fields are simulated and do not establish ADC quantisation behaviour.	Not tested

ID	Test case	Flash action	Actual/observed result	Verdict
TC11	UART auto-trigger reliability	Send one byte and a rapid burst on separate occasions.	A single accepted trigger produced one complete Sector 7 update; the rapid burst was consumed by the 50 ms quiet-window flush.	Pass
TC12	Power-cycle persistence	Reset after provisioning and result storage.	Identity and latest results were retained after a board reset; complete power-disconnection testing remains pending.	Partial

7.1 Terminal Output Evidence

The following terminal captures the Flash self test, identity provisioning, blank data handling, result replacement, reset persistence, and UART burst handling. Each caption identifies the corresponding verification case.

Terminal verifxcaton: Cases 1–4

```

screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
---- Previous Results (Sector 7) ----
No previous test data.
-----
Lab 04 Flash-only bare-metal build.
WARNING: Flash self-test erases Sector 7.
TC2/TC3 Flash self-test: OK

```

Case 1. Sector 7 erase and 0xDEADBEEF read/write self-test (TC2/TC3).

```

screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
Provisioning: OK
Rebuild with FLASH_PROVISION_IDENTITY_BUILD=0.

```

Case 2. First identity provisioning reports success (TC4).

```

screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
Provisioning: identity already provisioned
Rebuild with FLASH_PROVISION_IDENTITY_BUILD=0.

```

Case 3. Re-provisioning is rejected without erasing Sector 6 (TC4).

```

screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
---- Previous Results (Sector 7) ----
No previous test data.
-----
Lab 04 Flash-only bare-metal build.
Send any UART byte to erase/rewrite Sector 7 demo results.

```

Case 4. Erased Sector 7 produces the blank-results fallback (TC5/TC7).

Terminal verification: Cases 5–8

```
screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
---- Previous Results (Sector 7) ----
No previous test data.
-----
Lab 04 Flash-only bare-metal build.
Send any UART byte to erase/rewrite Sector 7 demo results.
Sector 7 update: OK
---- Previous Results (Sector 7) ----
12-bit: 1.047 V
10-bit: 1.048 V
8-bit: 1.053 V
6-bit: 1.072 V
-----
Send any byte to run again.
```

Case 5. First UART trigger stores and displays a result block.

```
screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
---- Previous Results (Sector 7) ----
12-bit: 1.394 V
10-bit: 1.395 V
8-bit: 1.400 V
6-bit: 1.419 V
-----
Lab 04 Flash-only bare-metal build.
Send any UART byte to erase/rewrite Sector 7 demo results.
```

Case 7. Identity and latest results are restored after reset.

```
screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
---- Previous Results (Sector 7) ----
No previous test data.
-----
Lab 04 Flash-only bare-metal build.
Send any UART byte to erase/rewrite Sector 7 demo results.
Sector 7 update: OK
---- Previous Results (Sector 7) ----
12-bit: 1.047 V
10-bit: 1.048 V
8-bit: 1.053 V
6-bit: 1.072 V
-----
Send any byte to run again.
```

Case 8. A rapid UART burst produces one complete update (TC11).

```
screen /dev/cu.usbmodem1103 115200

---- Student Identity (Sector 6) ----
Registration : 2023015944
Roll       : 01
Name       : MD.Jahidul Islam Sarker
-----
---- Previous Results (Sector 7) ----
No previous test data.
-----
Lab 04 Flash-only bare-metal build.
Send any UART byte to erase/rewrite Sector 7 demo results.
Sector 7 update: OK
---- Previous Results (Sector 7) ----
12-bit: 1.047 V
10-bit: 1.048 V
8-bit: 1.053 V
6-bit: 1.072 V
-----
Send any byte to run again.
Sector 7 update: OK
---- Previous Results (Sector 7) ----
12-bit: 1.394 V
10-bit: 1.395 V
8-bit: 1.400 V
6-bit: 1.419 V
-----
Send any byte to run again.
```

Case 6. Second trigger replaces the previous result block (TC6/TC9).

8 Analysis Questions (Assignment Section 8)

1. Why does Flash require an erase-before write cycle, while RAM does not?

A flash bit cell stores charge on a floating gate. Programming changes bits from 1 to 0, whereas returning programmed bits from 0 to 1 requires an erase operation applied to a whole sector. RAM cells are actively driven to either logical value and therefore do not require a separate erase-before-write cycle.

2. Estimate Sector 7's lifetime if re-triggered every few seconds all day, and propose a design change.

Assuming one trigger every 5 seconds and an endurance of approximately 10,000 erases, the lifetime is

$$10,000 \times 5 \text{ s} = 50,000 \text{ s} \approx 13.9 \text{ hours.}$$

At one trigger every 10 seconds, it is approximately 27.8 hours. Thus, erasing Sector 7 for every trigger can exhaust its rated endurance very quickly under continuous use. A better design would append multiple result records within the 128 KB sector and erase only when the sector becomes full (wear levelling), or commit only after a meaningful change or at a controlled interval.

3. Why store identity and test results in two separate sectors instead of one?

A sector erase removes every byte of this sector. If identity and results shared a sector, every results update would be erased the identity unless the identity were buffered and rewritten. Separating the data ensures that the normal Sector 7 update path cannot erase the write-once Sector 6 identity.

4. Compare the four resolutions' stored voltages; quantify the 1-LSB step size at each.

The theoretical 1-LSB step is $V_{REF}/\text{max_code}$:

$$\begin{aligned} \text{12-bit : } & 3.3/4095 \approx 0.81 \text{ mV,} & \text{10-bit : } & 3.3/1023 \approx 3.23 \text{ mV,} \\ \text{8-bit : } & 3.3/255 \approx 12.9 \text{ mV,} & \text{6-bit : } & 3.3/63 \approx 52.4 \text{ mV.} \end{aligned}$$

The displayed terminal values are deterministic simulation outputs used to test Flash storage; therefore, they do not constitute an observed ADC-resolution comparison. Such a comparison requires ADC sampling at a fixed potentiometer position.

5. Risk of losing power mid-erase/write, and how a more robust design could protect against corruption?

Power loss during erase can leave Sector 7 partially erased, and power loss during programming can leave an incomplete result payload. This implementation writes payload words first and the marker last, so an interrupted write normally lacks the valid marker and gets rejected at the next boot. A stronger design would maintain two result slots with sequence numbers and checksums, preserving the newest fully verified record when the other slot is being updated. CRC also can be used to achieve a robust design.

6. Compare register-level vs. HAL ADC_ReadRaw(): what does HAL do for us, and is there a cost?

Conceptually, a register-level implementation sets `SWSTART`, polls `EOC`, and reads `ADC1->DR` directly. HAL wraps these operations with handle-state checks, flag management, timeout handling, and standard error returns, adding a small execution and code size overhead.